

Reliability-Aware Fault Injection for Microscaling-Based DNNs

Project Report

Peeyush Agarwal

CS496 Undergraduate Project, Indian Institute of Technology Kanpur

7 October 2026

Abstract

Microscaling (MX) formats store a block of K narrow 4- to 8-bit elements with one shared 8-bit power-of-two scale. A bit fault can therefore land in an element, corrupting one value, or in the shared scale, rescaling all K values at once. This report characterises the reliability of MX-quantised DNNs under such faults. Using a bit-accurate fault-injection library, more than three million injections were run on ResNet8, RepVGG-A0 and a small vision Transformer, trained on CIFAR-10 and CIFAR-100, across five element formats, four block sizes, weights and activations, and every bit position, with two exhaustive campaigns as ground truth. Formats of equal accuracy differ in fault vulnerability by up to three orders of magnitude, with $\text{e3m2} < \text{e4m3} < \text{e5m2}$ on every network, and a scale fault is always more damaging than an element fault, typically by $13\times$ to over $1000\times$. The dominant mechanism is the NaN and infinity codes of the 8-bit formats: they carry 97–100% of element damage on the larger networks, model capacity never absorbs them, and decoding them as zero on read cuts element damage by $29\times$ or more. The study also shows that the conventional any-image SDC metric is confounded by near-tie inputs, and that exact enumeration of the MX code table reduces campaign cost by up to two orders of magnitude.

Keywords: DNN reliability, microscaling formats, MXFP, fault injection, shared scale, silent data corruption.

1 Introduction

MX formats [1, 2] make low-precision inference cheap by letting a block of K elements share one scale: the stored value is $v_i = \text{decode}(c_i) \cdot 2^{X-127}$, where c_i is a narrow element code and X an 8-bit E8M0 scale. In FP32 every value is a self-contained word, so a bit fault perturbs exactly one value, and existing DNN fault-injection work [3, 4, 5] rests on that assumption. MX breaks it: a fault can occur at two structurally different sites with very different blast radii.

CORE HYPOTHESIS

Reliability of microscaling DNNs varies substantially across formats **even at comparable inference accuracy**, and faults in the **shared scaling factor** behave very differently from faults in individual elements, because a single scale fault rescales an entire block of K values at once.

The study has five objectives: **O1** characterise element versus scale fault propagation; **O2** compare formats at equal accuracy; **O3** measure the block-size trade-off; **O4** compare weights with activations and fault locations; **O5** identify the representation characteristics that drive vulnerability. Two methodological choices are settled along the way: learned value intervals [6] versus exact code enumeration, and single-bit versus multi-bit injection.

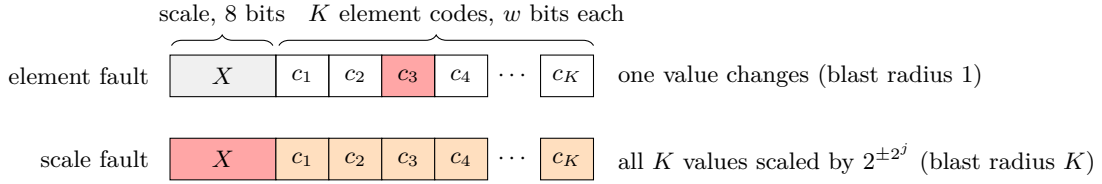
2 Background

Formats. The scale is E8M0: an unsigned power of two with bias 127, code 255 being NaN. Elements are small sign–exponent–mantissa floats (Table 1). Only the two 8-bit formats reserve bit patterns for NaN or infinity, which turns out to be the property that matters most. The OCP rule sets $X - 127 = \lfloor \log_2 \max_i |v_i| \rfloor - e_{\max}$, aligning the block maximum with the format’s top binade; since the largest element lies below the top of that binade, the maximum can be clipped by up to 12.5% (25% for e2m1).

Table 1: MX element formats. The 6- and 4-bit formats have no NaN/Inf codes.

format	family	sign/exp./mantissa	bias	largest value	NaN/Inf codes
e4m3	MXFP8	1/4/3	7	448	2 (NaN)
e5m2	MXFP8	1/5/2	15	57,344	8 (2 Inf, 6 NaN)
e3m2	MXFP6	1/3/2	3	28	none
e2m3	MXFP6	1/2/3	1	7.5	none
e2m1	MXFP4	1/2/1	1	6	none

Two fault sites. An **element fault** flips bit b of one code: a mantissa bit nudges the value, an exponent bit scales it, the sign bit negates it, and in e4m3/e5m2 the flip may produce NaN or infinity. A **scale fault** flips bit j of X and multiplies all K values by $2^{\pm 2^j}$; bit 7 shifts the exponent by 128 and overflows float32 (Figure 1). Scales are only $8/(8 + Kw)$ of storage, about 3% at $K = 32$, $w = 8$, yet under uniform bit upsets they receive $8 : w$ of all corrupted values.

**Figure 1:** One MX block and its two fault sites. Red marks the flipped word and orange the values it alters.

Metrics. For a fault φ and a fixed evaluation set of N images, let $\hat{y}_i$ be the golden prediction of the fault-free quantised network. The conventional *any-image SDC rate* is the fraction of faults that change at least one of the N predictions. The *per-inference rate* is the mean over faults of the fraction of the N predictions that change; it answers how likely one fault is to corrupt a given inference. Section 5 shows the any-image rate is confounded by the evaluation set, so **all rates below are per inference**. Proportions carry Wilson 95% intervals [10].

3 Methodology

Injection engine. The `mxfi` library encodes each tensor into real MX bit patterns, with scales and element codes as separately addressable arrays, applies a fault as an XOR of one bit, and decodes. Because an element alphabet has at most 256 codes, the decoded change of every (code, bit) flip is also tabulated exactly. Networks are injected in deployment form (batch norm folded, RepVGG branches fused); weights are blocked along the reduction axis. Following the OCP conversion, a NaN anywhere in a block makes the block NaN, so NaNs propagate through activation re-quantisation. Activation faults are injected through forward hooks into one image’s encoding. The library has 214 unit tests.

Networks. ResNet8 (MLPerf Tiny [7], 78K parameters, also at five widths spanning 20K–692K with up to ten seeds per width), RepVGG-A0 [8] (7.0M fused), and a ViT at DeiT-Tiny width [9] (2.7M, depth 6), each trained on CIFAR-10 (87.0%, 91.4%, 80–82%) and CIFAR-100 (58%, 70–71%, 47–49%) with three seeds. After MX quantisation at $K = 32$ the 6- and 8-bit formats are within about one point of each other on every network, and within 0.2 points of FP32 on the ViT, so formats can be compared at matched accuracy.

Campaigns. Each fault is scored on a fixed, class-balanced set of 200 test images. Most campaigns draw 3000 faults uniformly over all stored bits. Two *exhaustive* campaigns flip every one of the 638,624 (e4m3) and 483,904 (e3m2) weight bits of a ResNet8; a 3000-fault estimate lands within 2.4% of the exhaustive rate and its interval covers it. Every campaign is tied to its checkpoint by replaying recorded faults. In total the study ran over three million injections on more than sixty trained networks, on a CPU and two NVIDIA A100 GPUs.

4 Results

4.1 O2: Equal accuracy, very different reliability

Within about a point of accuracy, element-fault damage spans more than two orders of magnitude on ResNet8 and nearly four on RepVGG and the ViT (Figure 2). On the ViT, e5m2 and e3m2 differ by 0.01 points of accuracy, yet an e5m2 element fault is about $2000\times$ as likely to corrupt an inference. The ordering $e3m2 < e4m3 < e5m2$ holds on all eight CIFAR-10 configurations with disjoint intervals (Table 2) and on all nine CIFAR-100 networks.

Within about a point of accuracy, fault vulnerability spans up to three orders of magnitude

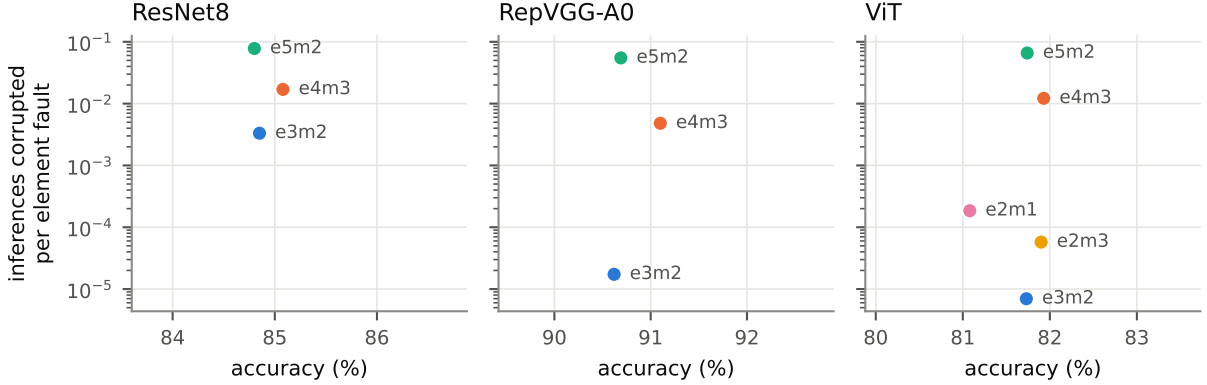


Figure 2: Quantised accuracy against per-inference damage of an element fault (CIFAR-10, $K = 32$). Each point is one format.

Table 2: Per-inference rate of element faults, CIFAR-10, $K = 32$, mean over seeds.

network	seeds	e3m2 (6-bit)	e4m3 (8-bit)	e5m2 (8-bit)
RepVGG-A0	3	0.00001	0.00688	0.05381
ViT	3	0.00003	0.01258	0.06897
ResNet8	1	0.00332	0.01699	0.07801
ResNet8, width 8	5	0.01003	0.02005	0.08143
ResNet8, width 16	5	0.00180	0.01374	0.07831
ResNet8, width 24	4	0.00121	0.01210	0.07585
ResNet8, width 32	4	0.00060	0.01354	0.07934
ResNet8, width 48	4	0.00035	0.01302	0.07812

4.2 O1: Shared-scale faults versus element faults

A scale fault is more damaging than an element fault on every network and format, from $1.7\times$ for e5m2, whose element faults are already NaN-dominated, to over $3000\times$ for e3m2 on the larger networks (Figure 3). In the exhaustive campaigns the scales are 4.1% of e3m2 storage but carry 87.6% of all damage, and 3.1% of e4m3 storage carrying 32.7%. About 12% of scale faults give a non-finite output in every format, since E8M0 has its own NaN code. Flips that grow a block are the dangerous ones: on ResNet8, scale bit 3 (multiplying the block by 256) corrupts 75% of inferences and bit 7 (overflow) 98%, while bits 4–6, which shrink the block, corrupt 3%.

4.3 O3: Block size

Larger blocks shrink the scale’s share of storage from 11.1% ($K = 8$) to 1.7% ($K = 64$) but make each scale fault worse, while element faults become milder (Figure 4). The element-side trend appears under both the OCP and a non-clipping scale rule, and accuracy moves by under a point across K . The two sites move in opposite directions, so block size is a genuine trade-off; with protected scales, large blocks are safe.

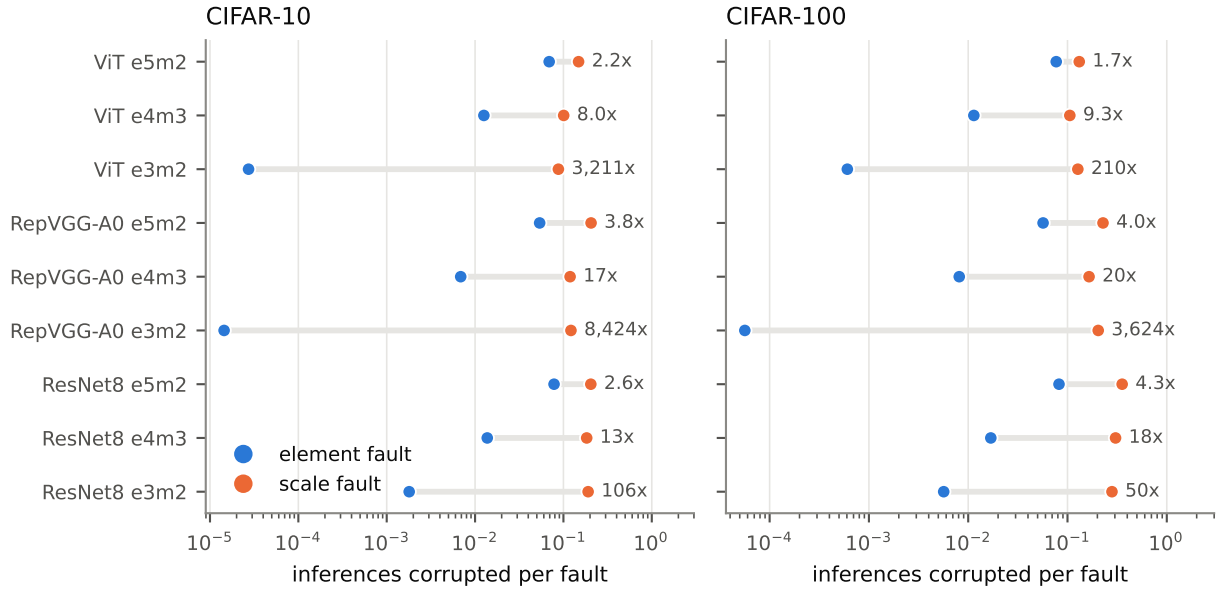


Figure 3: Per-inference damage of element (blue) and scale (orange) faults, with their ratio, mean over seeds.

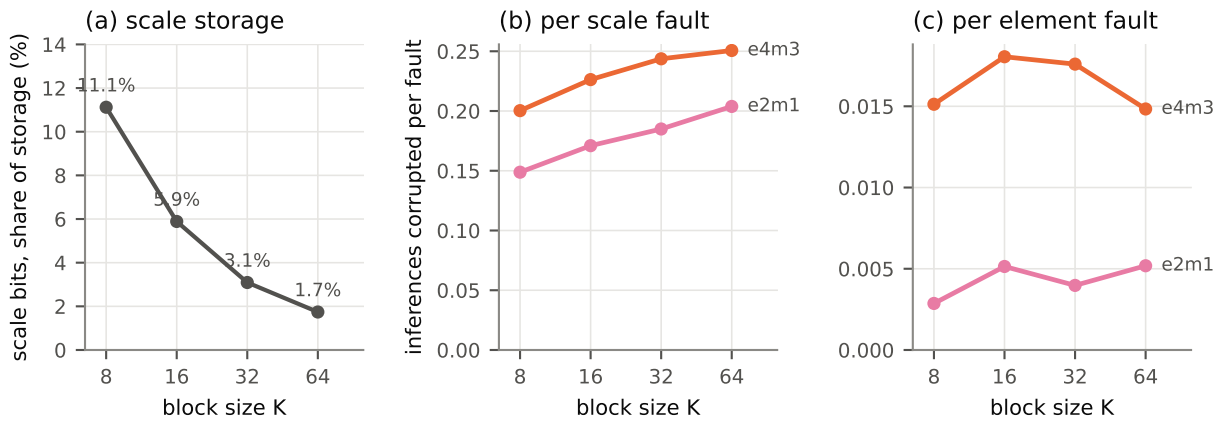


Figure 4: Block size on ResNet8: (a) scale share of storage, (b) damage per scale fault, (c) damage per element fault.

4.4 O4: Bit position, layer and tensor type

Among faults that leave the output finite, the sign bit is the most damaging element bit; in total damage it ranks seventh of eight in e4m3 (Figure 5). The 1.27% of element faults that flip a code into the NaN pattern `S.1111.111` corrupt every inference and carry 77.6% of all element damage, and a sign flip is the one flip that cannot reach it. Vulnerability runs opposite to layer size: the 3584-bit stem convolution is the most fragile layer, the 304,128-bit final convolution the most robust. Per fault, weights and activations are about equally dangerous ($0.3\text{--}2.2\times$ across six networks and two formats), because about 1% of faults in either land on a special code; they differ only in exposure, a weight fault persisting across inferences.

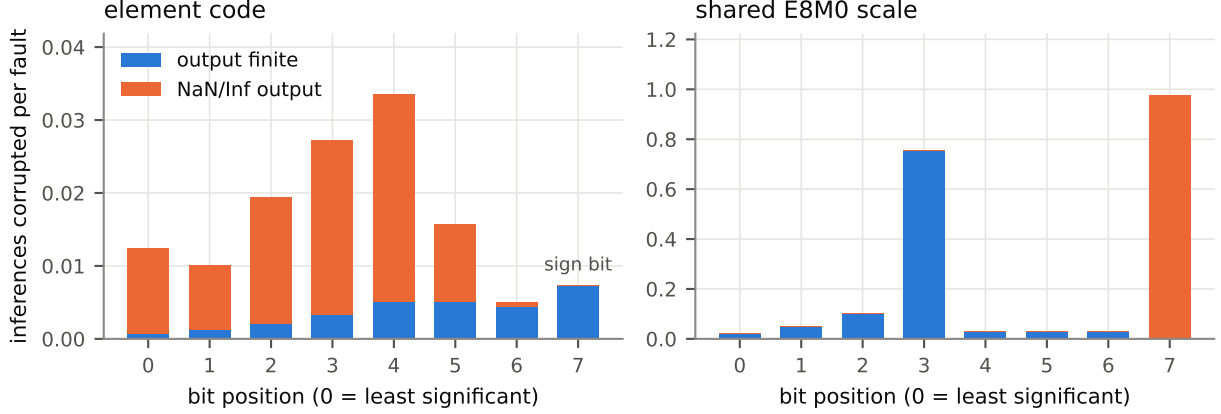


Figure 5: Exhaustive per-bit damage (ResNet8, e4m3, $K = 32$, 638,624 injections), split by whether the output stayed finite.

4.5 O5: What drives vulnerability

NaN and infinity codes. A larger network absorbs ordinary perturbations but never a NaN, and only formats with special codes can produce one. Across the ResNet8 width sweep, a $35\times$ range of parameters, capacity improves e3m2 (no special codes) $25\times$, e4m3 (two) $1.5\times$ and e5m2 (eight) not measurably (Figure 6); the fraction of faults that produce NaN/Inf stays flat with width. On RepVGG and the ViT, special-code faults carry 97–100% of e4m3 and e5m2 element damage.

Margin-normalised sensitivity. For the remaining perturbation damage, define the sensitivity of a weight as $s_i = |\partial m / \partial w_i| \cdot \text{rms} / m$, where m is the top-1/top-2 logit margin. It predicts a format-agnostic weight-perturbation probe at $r = +0.998$, and perturbation damage across all five formats at $r = +0.93$, while mantissa width does not ($r = -0.25$; Figure 7). The gradient factor agrees across formats to $1.08\times$; the spread comes from the margin of the evaluation image nearest a decision boundary.

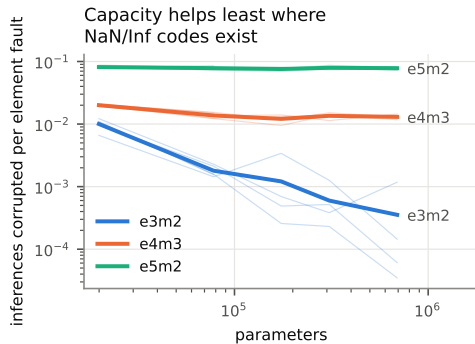


Figure 6: Element-fault damage against ResNet8 width; faint lines are seeds.

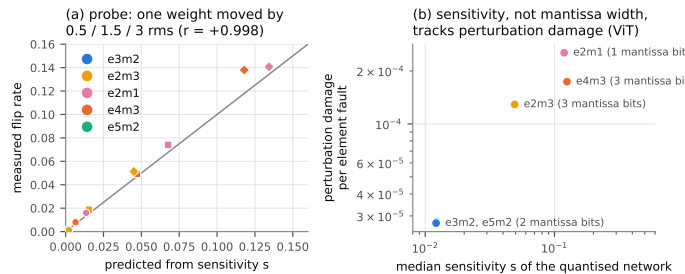


Figure 7: (a) s predicts a weight-perturbation probe. (b) Perturbation damage follows sensitivity, not mantissa width (ViT).

4.6 A read-side guard for special codes

Decoding any special element code as zero on read turns a NaN-producing fault into a single zeroed weight. Measured on every architecture with e4m3 and e5m2 (Table 3), the guard cuts element damage by at least 29 $\times$ and up to 4100 $\times$ on RepVGG and the ViT, bringing e5m2 to the level of e3m2, and by 4–53 $\times$ on ResNet8, which is also fragile to ordinary perturbations. No guarded fault produced a non-finite output. In hardware it is one comparator on the read path.

Table 3: Effect of decoding special element codes as zero on read (CIFAR-10). Reductions are lower bounds (unguarded rate over a 95% upper bound on the guarded rate).

network	format	element damage from special codes	reduction
ResNet8 (2 nets)	e4m3 / e5m2	78–93% / 98–99%	$\geq 3.9\times$ / $\geq 30\times$
RepVGG-A0 (3 seeds)	e4m3 / e5m2	99.5–99.9% / 99.9–100%	$\geq 115\times$ / $\geq 319\times$
ViT (3 seeds)	e4m3 / e5m2	97.1–99.5% / 99.9–100%	$\geq 29\times$ / $\geq 43\times$

4.7 CIFAR-100

CIFAR-100 stands in for ImageNet. Every result above replicates. With smaller margins, the NaN-free formats become 3–22 $\times$ more vulnerable while e4m3 and e5m2 barely move (Figure 8), so the gap narrows (on the ViT, e4m3/e3m2 falls from 461 $\times$ to 19 $\times$). The guard still gives 13–1387 $\times$ on RepVGG and the ViT.

A harder task raises the NaN-free formats; the NaN-coded ones barely move

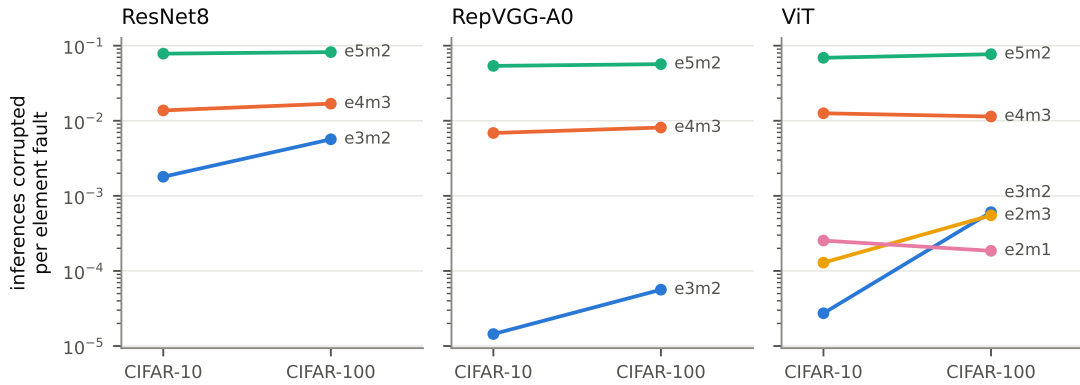


Figure 8: Element-fault damage on CIFAR-10 and CIFAR-100, mean over seeds.

5 Methodological findings

The any-image SDC metric measures the evaluation set. One fixed fault list replayed against nine disjoint 200-image sets on two networks makes e3m2 and e4m3 look equal under the any-image metric (1.03 $\times$, 0.93 $\times$), with their order reversing between sets; per inference they differ 5.6 $\times$ and 7.6 $\times$, stable to within 1.3 $\times$ (Figure 9). The any-image rate saturates and mostly reports whether the set contains a near-tie input (Spearman +0.89 with the near-tie count over 39 networks). Separately, networks differing only in their training seed differ by a median 12.8 $\times$ the injection uncertainty, so results must be replicated over trained networks.

Severity. The log-ratio severity $|\log v' - \log v|$ scores a sign flip as zero and ranks failures below chance (AUC 0.435). The margin-based $\text{margin_shift}_i = s_i |v'_i - v_i| / \text{rms}$ reaches AUC 0.990; flagging $\text{margin_shift} > 1$ catches 98.5% of failures in 4.4% of the fault space.

Value-aware sampling. The per-inference target is zero-inflated (92% of e3m2 element faults corrupt nothing). Stratifying at quantiles of margin_shift with Neyman allocation [11] cuts variance 57–120 $\times$ against uniform sampling, replayed against exhaustive ground truth. Exact enumeration of the decoded change alone gives 48–65 $\times$, while TreeFI-style intervals learned from a pilot never beat uniform ($\leq 1\times$): a pilot cannot see the tail, and in e4m3 the tail is a discrete set of NaN transitions rather than a value range.

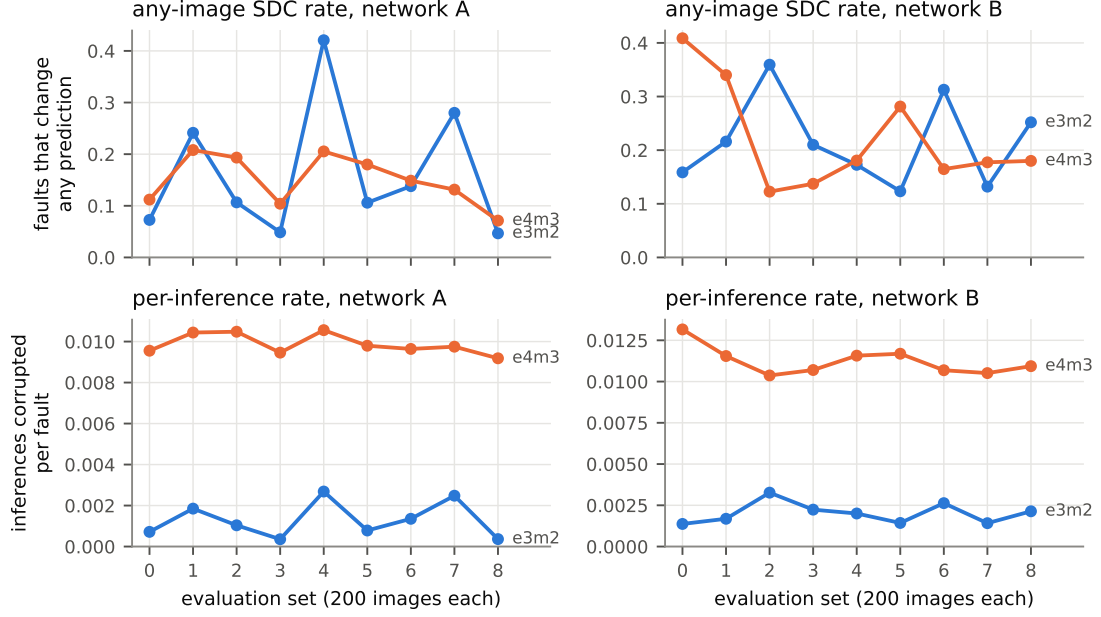


Figure 9: One fault list per format replayed on nine evaluation sets, two networks: any-image rate (top) against per-inference rate (bottom).

Single versus multi-bit. Injecting every single and double flip of 2000 words per format on two networks (390,000 injections), a double flip does $1.04\text{--}1.55\times$ the damage of a single flip and an adjacent pair $0.85\text{--}1.13\times$. Within-word doubles reach 1% of expected damage only at a per-bit flip probability of $2\text{--}3 \times 10^{-3}$, far beyond where the one-fault-per-inference assumption holds. Single-bit injection is sufficient.

6 Summary and design guidance

Table 4: The five objectives and their answers (per-inference rates).

objective	answer
O1 Fault propagation	Scale faults are always more damaging, typically $13\times$ to over $1000\times$; about 12% give non-finite outputs in every format.
O2 Format sensitivity	Equal-accuracy formats differ by up to three orders of magnitude; $e3m2 < e4m3 < e5m2$ on all 17 networks.
O3 Block size	Larger blocks make scale faults rarer but worse and element faults milder.
O4 Tensor & location	Weights and activations equally dangerous per fault; NaN transitions dominate element damage; the stem layer is most fragile.
O5 Drivers	NaN/Inf codes first (97–100% of element damage on larger networks); then the network’s margin-normalised sensitivity, not mantissa width.

For MX accelerator designers:

1. **Protect the shared scale first:** 2–11% of storage, $13\times$ to over $1000\times$ the damage; protect first the scale bits that are normally 0.
2. **Neutralise special codes:** prefer NaN-free element formats, or decode special codes as zero on read.
3. **Do not judge reliability by accuracy or bit width.**
4. **Treat block size as a trade-off,** and protect by vulnerability (early layers), not by storage size.

For fault-injection campaigns: report per-inference rates, replicate over trained networks, inject single-bit faults, and stratify by exact per-code enumeration weighted by sensitivity.

Limitations. ImageNet was not available on the compute used, so a ViT trained from scratch on CIFAR-100 stands in for DeiT on ImageNet; the directions of the effects transfer more reliably than their magnitudes.

The largest network has 7M parameters, only faults in stored values are modelled, and the guard is evaluated in software.

7 Conclusion

Reliability of MX networks varies by up to three orders of magnitude across formats of equal accuracy, and shared-scale faults are always more damaging than element faults. Most element damage comes from the few flips that land on NaN or infinity codes, which capacity cannot absorb and a read-side guard removes; the rest follows the network’s decision margins. The work also provides a validated MX fault-injection library, a per-inference metric that avoids the near-tie confound, and a sampling method that cuts campaign cost by up to two orders of magnitude.

Code and data. The `mxfi` library, experiment scripts, trained networks and raw results are at https://github.com/peeyushagarwal2004/CS496_UGP.

Acknowledgements. I am grateful to my supervisor for the problem statement and guidance, and to IIT Kanpur for access to its GPU server.

References

- [1] B. D. Rouhani et al., “Microscaling data formats for deep learning,” *arXiv:2310.10537*, 2023.
- [2] Open Compute Project, “OCP Microscaling Formats (MX) Specification, Version 1.0,” 2023.
- [3] G. Li et al., “Understanding error propagation in deep learning neural network (DNN) accelerators and applications,” in *Proc. SC*, 2017.
- [4] B. Reagen et al., “Ares: A framework for quantifying the resilience of deep neural networks,” in *Proc. DAC*, 2018.
- [5] A. Mahmoud et al., “PyTorchFI: A runtime perturbation tool for DNNs,” in *Proc. DSN Workshops*, 2020.
- [6] TreeFI: value-aware statistical fault injection for DNNs, in *Proc. ICCAD*, 2026.
- [7] C. Banbury et al., “MLPerf Tiny benchmark,” in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [8] X. Ding et al., “RepVGG: Making VGG-style ConvNets great again,” in *Proc. CVPR*, 2021.
- [9] H. Touvron et al., “Training data-efficient image transformers & distillation through attention,” in *Proc. ICML*, 2021.
- [10] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. American Statistical Association*, vol. 22, 1927.
- [11] J. Neyman, “On the two different aspects of the representative method,” *J. Royal Statistical Society*, vol. 97, 1934.